AGENTGROUPCHAT-V2 : Divide-and-Conquer Is What LLM-Based Multi-Agent System Need

Zhouhong Gu[*,1], **Xiaoxuan Zhu**[*,1], **Yin Cai**[2], **Hao Shen**[1], **Xingzhou Chen**[1], **Qingyi Wang**[1], **Jialin Li**[1], **Xiaoran Shi**[1], **Haoran Guo**[3], **Wenxuan Huang**[4], **Hongwei Feng**[1], **Yanghua Xiao**[1], **Zheyu Ye**[5], **Yao Hu**[5], **Shaosheng Cao**[5]
[1]Shanghai Key Laboratory of Data Science, School of Computer Science, Fudan University
[2]Fudan University, [3]Rhine AI, [4]East China Normal University, [5]Xiaohongshu
* Equal Contribution
{zhgu22, xxzhu22, haoshen22, xzchen24, xrshi21, wangqingyi22, jialingli22}@m.fudan.edu.cn
{tanghuang1, guoba1}@xiaohongshu.com, {yincai, shawyh, hwfeng}@fudan.edu.cn

arXiv:2506.15451v1 [cs.CL] 18 Jun 2025

***Abstract*—Large language model based multi-agent systems have demonstrated significant potential in social simulation and complex task resolution domains. However, current frameworks face critical challenges in system architecture design, cross-domain generalizability, and performance guarantees, particularly as task complexity and number of agents increases. We introduces AGENTGROUPCHAT-V2 , a novel framework addressing these challenges through three core innovations: (1) a divide-and-conquer fully parallel architecture that decomposes user queries into hierarchical task forest structures enabling dependency management and distributed concurrent processing. (2) an adaptive collaboration engine that dynamically selects heterogeneous LLM combinations and interaction modes based on task characteristics. (3) agent organization optimization strategies combining divide-and-conquer approaches for efficient problem decomposition. Extensive experiments demonstrate AGENTGROUPCHAT-V2 's superior performance across diverse domains, achieving 91.50% accuracy on GSM8K (exceeding the best baseline by 5.6 percentage points), 30.4% accuracy on competition-level AIME (nearly doubling other methods), and 79.20% pass@1 on HumanEval. Performance advantages become increasingly pronounced with higher task difficulty, particularly on Level 5 MATH problems where improvements exceed 11 percentage points compared to state-of-the-art baselines. These results confirm that AGENTGROUPCHAT-V2 provides a comprehensive solution for building efficient, general-purpose LLM multi-agent systems with significant advantages in complex reasoning scenarios. Code is available at https://github.com/MikeGu721/AgentGroupChat-V2.**

I. INTRODUCTION

In recent years, research on LLM-based (large language model) multi-agent systems has grown substantially (Guo et al., 2024; Li et al., 2023; Xi et al., 2023; Liang et al., 2024). These systems demonstrate considerable potential in two primary application domains: social simulation (Gao et al., 2024, 2023; Gu et al., 2024; Park et al., 2023; Zhou et al., 2023) and complex task resolution (Hong et al., 2023; Wu et al., 2023; Qian et al., 2024; Xiong et al., 2023).

In social simulation scenarios, these systems facilitate the modeling of human-like social interactions and behavioral patterns (Park et al., 2023; Bates et al., 1994), providing valuable insights for sociological research, policy development, and risk assessment (Lee et al., 2023; Zhang et al., 2024). In task-oriented scenarios, collaborative agent networks demonstrate enhanced complex reasoning, planning (Huang et al., 2024; Wei et al., 2022), and creative problem-solving capabilities through distributed intelligence (Du et al., 2023). Compared to traditional single-agent systems (Chen et al., 2024; Qin et al., 2023), multi-agent architectures not only enable the integration of complementary knowledge (Wooldridge, 2009), but also generate emergent problem-solving strategies through structured inter-agent interactions and hierarchical collaboration (Ferber & Weiss, 1999), thereby achieving superior collective intelligence (Dafoe et al., 2021).

However, as the application scope of LLM multi-agent systems expands and task complexity increases, existing frameworks face systematic challenges (Guo et al., 2024; Ouyang et al., 2022; Cemri et al., 2025). First, existing solutions are predominantly designed for specific domains: systems like S3 (Gao et al., 2023) and AgentGroupChat (Gu et al., 2024) focus on social simulation scenarios, while MetaGPT (Hong et al., 2023) and AutoGen (Wu et al., 2023) target specific tasks such as software development, lacking general-purpose design for handling diverse problem types (Shoham & Leyton-Brown, 2008). Second, multi-agent interactions often involve large-scale LLM calls and complex agent coordination, with existing systems commonly adopting sequential execution patterns, resulting in substantial computational resource waste and time overhead (Kumar, 2025; Zhu, 2025). Finally, although multi-agent collaboration employs greater computational resources and should theoretically generally outperform single-agent approaches in terms of effectiveness, this is often not the case in practical scenarios (Zhang et al., 2023; Luo et al., 2025; Cemri et al., 2025).

The fundamental issue underlying the aforementioned challenges lies in the prevalent lack of effective collaboration schemes for general-purpose scenarios in existing LLM-based multi-agent systems. We argue that divide-and-conquer represents a key approach to addressing this fundamental problem. The divide-and-conquer strategy encompasses not only the decomposition and processing of complex tasks, breaking down large tasks into a series of manageable subtasks, but also the

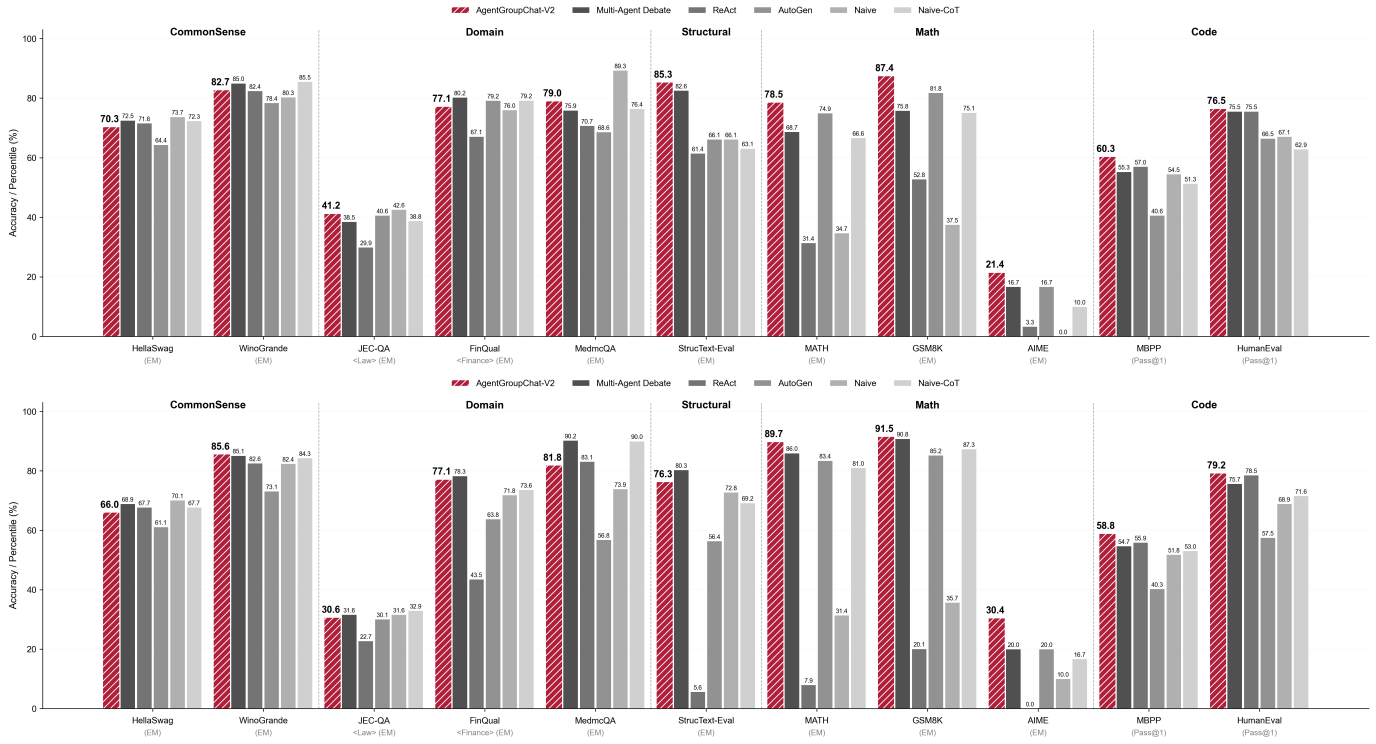

Fig. 1. **Upper:** The performance of Qwen2.5-72B. **Downer:** The performance of Llama3.1-70B. Both models are evaluated across five diverse reasoning domains: commonsense reasoning, domain-specific knowledge, structural text understanding, mathematics, and code generation. AGENTGROUPCHAT-V2 consistently outperforms existing multi-agent approaches and baseline methods across both models and all benchmark categories, demonstrating the effectiveness, robustness and promising of multi-agent approach.

optimization and decomposition of multi-agent collaboration itself, enabling each agent to focus on specific aspects of the task. This approach can significantly enhance task processing effectiveness and parallel execution efficiency, while effectively improving the overall collaborative performance of multi-agent systems through specialized division of labor among agents, thereby providing a viable pathway for constructing general-purpose multi-agent collaboration schemes (Shoham & Leyton-Brown, 2008; Ferber & Weiss, 1999).

To construct truly practical LLM multi-agent systems, three core challenges must be systematically addressed: (1) System architecture challenge: LLM multi-agent systems are inherently resource-intensive applications, yet current systems lack architectures specifically designed to support large-scale multi-agent collaboration (Ferber & Weiss, 1999; Kumar, 2025); (2) Generalizability challenge: Existing systems primarily employ customized designs for specific scenarios, lacking cross-domain adaptability and adaptive problem understanding capabilities (Shoham & Leyton-Brown, 2008; Gao et al., 2023); (3) Performance guarantee challenge: Many multi-agent systems fail to demonstrate clear performance advantages over single-agent approaches in practical applications, sometimes performing worse due to system complexity (Wooldridge, 2009; Zhang et al., 2023).

The core innovations of AGENTGROUPCHAT-V2 include: 1) A fully parallel architecture with three coordinated manager modules that enable efficient resource control and high-throughput operation of LLM-based multi-agent systems, supporting distributed cluster deployment where manager modules coordinate global tasks on central servers while group managers handle parallel processes across multiple servers, ensuring distributed concurrent processing at all LLM invocation points and significantly improving system throughput and resource utilization; 2) Task-level divide-and-conquer through dynamic task tree decomposition, where complex queries are hierarchically broken down into manageable subtasks with optimized dependency management and parallel execution; 3) Execution-level divide-and-conquer through specialized agent role assignment, where different LLMs assume distinct roles and focus on specific aspects of problem-solving, enabling adaptive collaboration and cross-domain generalization. Experimental results demonstrate that AGENTGROUPCHAT-V2 achieves significant performance improvements while maintaining system generalizability, with particularly outstanding advantages on high-difficulty tasks (Figure 1), delivering substantial improvements across mathematical reasoning, code generation, and complex reasoning scenarios (Qin et al., 2023; Park et al., 2023; Hong et al., 2023; Wu et al., 2023).

II. RELATED WORK

LLM-based multi-agent systems have demonstrated transformative potential in two key domains Li et al. (2023); Wooldridge (2009):

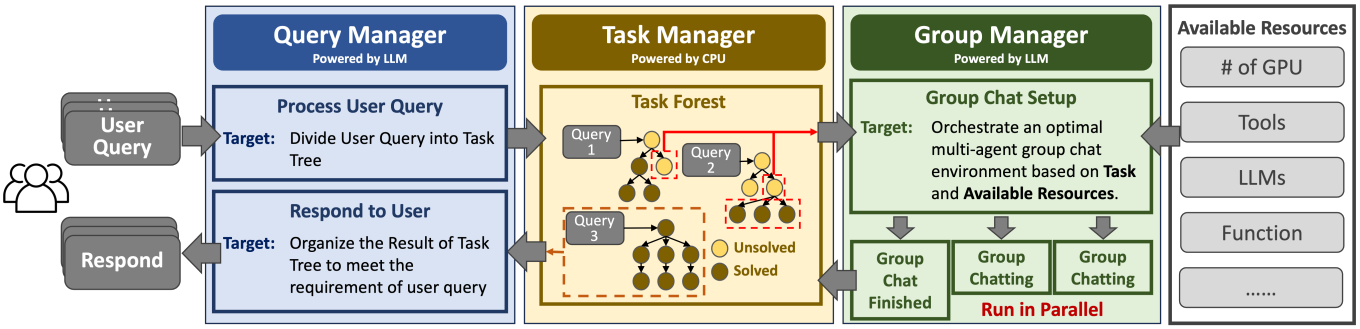

Fig. 2. Illustration of AgentGroupChat-V2 framework, which composes of three main components: Query Manager, Task Manager and Group Manager. The framework illustrates the complete workflow from user query processing through task decomposition and management to multi-agent group chat execution, with arrows indicating data flow between components. Task Forest visualization demonstrates how queries are transformed into hierarchical task structures with solved (brown) and unsolved (yellow) nodes, while parallel group chats are carrying out in Group Manager.

A. Social Dynamics Simulation

Recent advances integrate cognitive architectures with behavioral economics principles Bates et al. (1994).Xie et al. Xie et al. (2024) and Han et al. Han et al. (2023) develop game-theoretic frameworks capturing trust dynamics and market competition through iterative belief-updating mechanisms Dafoe et al. (2021).Sociological simulations leverage event-driven architectures, with Park et al. Park et al. (2023) modeling opinion evolution through social interaction cascades, while Zhang et al. Zhang et al. (2024) employ hierarchical Bayesian networks for electoral behavior prediction Lee et al. (2023). Emerging platforms like Gu's group chat simulator Gu et al. (2024) and Liu's rumor-control Twitter emulator Liu et al. (2024) demonstrate practical applications in digital social dynamics Gao et al. (2023).

B. Collaborative Problem-Solving

Cutting-edge systems employ structured debate protocols and knowledge fusion mechanisms Du et al. (2023).Xiong's FORD framework Xiong et al. (2023) enhances reasoning through tri-phase argumentation processes, complemented by Du's knowledge graph-based consensus formation Du et al. (2023).Software engineering innovations like Qian's dialogue-driven development Qian et al. (2024) and Hong's documentation-centric workflow Hong et al. (2023) establish new paradigms for AI-assisted programming Wang et al. (2023).Domain-specific implementations showcase methodological cross-pollination, from Sun's legal argumentation system with adversarial validation Sun et al. (2024) to Yu's cognitive conflict-driven MOOC platform Yu et al. (2024), demonstrating versatile problem-solving architectures Ferber & Weiss (1999).

III. Framework of AgentGroupChat-V2

Building upon the AgentGroupChat framework Gu et al. (2024), this research presents AgentGroupChat V2. As illustrated in Figure 2, AgentGroupChat-V2 employs a tripartite architecture comprising Query Manager, Task Manager, and Group Manager. This modular structure facilitates data exchange and function invocation through standardized interfaces and communication protocols. The architectural design of AgentGroupChat-V2 enables independent scaling of each module according to computational requirements, supporting horizontal expansion of the system's processing capabilities.

A. Query Manager

Query Manager functions as the frontend interaction component, integrating large language models as inference engines for direct user interaction. Its core functions include receiving User Queries, decomposing them into task tree structures through semantic understanding and task analysis, and forwarding these structures to the Task Manager for processing. Upon task completion, Query Manager receives the processing results, standardizes and integrates them, ensuring the response content meets user query requirements and is presented in an appropriate format.

B. Task Manager

Task Manager serves as the central coordination component, powered by CPU, implementing task flow management. A typical AgentGroupChat-V2 system is configured with a single Task Manager responsible for maintaining the state and execution of the entire task forest (containing multiple task trees derived from user queries). This module constructs a comprehensive task tracking system that records bidirectional relationships between tasks—child tasks link to their parent tasks, and parent tasks map to all their child tasks, forming a complete task dependency graph. Upon receiving tasks allocated by the Query Manager, the Task Manager determines task distribution strategies based on task characteristics and system load conditions. For hierarchically structured task groups, Task Manager implements information flow management, ensuring that processing results from child tasks serve as background knowledge for parent task processing workflows. For tasks without dependencies, Task Manager can distribute them to Group Manager instances based on resource availability, enabling parallel processing. When tasks are completed, Task Manager updates the task tree status, and when all tasks in a task tree have obtained results, the entire task tree is transmitted to Query Manager.

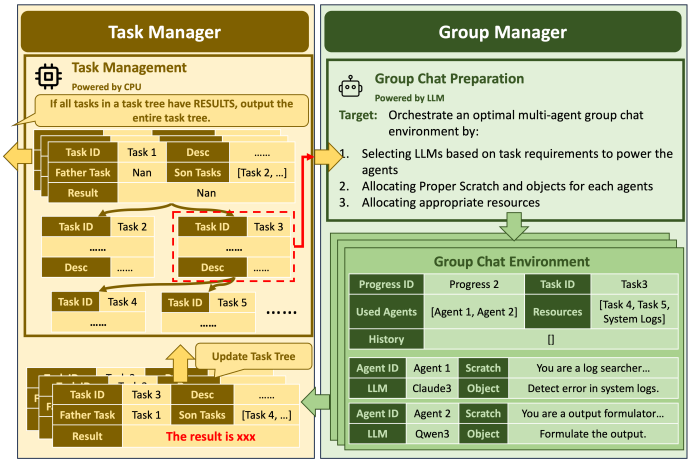

Fig. 3. Detailed implementation of Task Manager and Group Manager modules showing: (1) Task Management structure with hierarchical organization of tasks, including Task ID, descriptions, parent-child relationships, and result tracking; (2) Group Chat Preparation process outlining agent selection criteria and resource allocation strategies; and (3) Group Chat Environment configuration displaying the agent infrastructure with Claude3 and Qwen3 LLMs assigned specific roles, workspace allocations, and specialized functions for systematic problem-solving through collaborative interactions.

C. Group Manager

Group Manager functions as the execution component, responsible for organizing and coordinating multi-agent collaborative activities to accomplish specific tasks. This module incorporates large language models, with its primary function occurring during the group chat preparation phase, including selecting appropriate LLMs as inference engines for agents based on task requirements, allocating suitable workspace and object resources for each agent, and efficiently scheduling computational resources. Group Manager can be scaled according to system load, with multiple instances operating in parallel, each managing independent agent groups, enabling efficient utilization of system computational resources.

IV. Group Chat Design

This section elaborates on the organizational implementation mechanisms of group chat in the AgentGroupChat-V2 system. As the core element of multi-agent collaboration, the design of group chat directly impacts the system's problem-solving capabilities. As shown in Figure 3, the system employs structured approaches to task organization and agent grouping to facilitate effective multi-agent problem solving.

A. Task

In the AgentGroupChat-V2 system, Task serves as the basic unit of task processing with a clearly structured definition. As shown in Figure 3, each Task includes core attributes: Task ID (unique identifier), description information (Desc), parent task association (Father Task), child task collection (Son Tasks), and processing result (Result). When constructing task trees, the system marks the parent task of a root task as "Nan," while leaf tasks have empty child task collections. Tasks in the system follow a state transition process: initialization state (creating task structure without allocating execution resources) → waiting state (awaiting completion of prerequisite dependent tasks) → execution state (assigned to Group Manager for processing) → completion state (processing result generated) or failure state (errors encountered during processing).

B. Group

In the AgentGroupChat-V2 system, Group represents a multi-agent collaborative work unit created and managed by Group Manager. The Group Chat Environment shown in Figure 3 illustrates the core structure of a Group. Each Group contains basic information including progress identifier (Progress ID), participating agent list (Used Agents), task association (Task ID), and related resources (Resources). Each agent (Agent) within a Group possesses a clear identity (Agent ID), inference engine (LLM, typically large models such as Claude3 or Qwen3), dedicated workspace (Scratch), specific objects (Object), and a History field for storing interaction records. The Scratch space is primarily for the LLM's internal reference, while Object information is designed for human oversight, describing the agent's specific role and function (such as log searching or error detection). The History component stores the agent's conversation history, enabling persistent memory across multiple interaction rounds. Group Manager constructs complementary collaborative teams by assigning different functional positions and resource permissions to different agents. A Group's lifecycle includes: preparation phase (Group Manager selects and configures agents) → activity phase (agents engage in multi-turn dialogue to solve problems collectively) → result integration phase (transforming discussion outcomes into task outputs) → termination phase (releasing resources and updating system status).

C. Group Environment Configuration

Group environment configuration forms the foundational condition for effective group chat. Each group environment contains a unique Progress ID to identify the current progress, an associated Task ID indicating which task the group is responsible for processing, Used Agents recording the set of participating agents and their speaking order in the group chat, and a Resources field defining available system resources shared across the group, such as execution records of specific tasks or system logs.

The group environment configuration process is led by Group Manager. Based on task requirements transmitted by Task Manager, Group Manager makes intelligent decisions through its built-in large language model, selecting the most suitable inference engine for each agent according to available LLM types and their relevance to the task. This design allows the system to flexibly configure heterogeneous LLM combinations based on different task characteristics, while also allowing different identities to be assigned to LLMs, enabling each Agent to focus on a specific objective. Group Manager configures three key elements for each agent: Scratch space for granting the LLM specific identity cognition; Object information for recording the agent's specific tasks, providing task transparency

Algorithm 1 StartGroupChat (GroupManager)

```
Input: max_action_turn, agent_ids, initial_env
Output: final_env, task_result
env ← initial_env
for turn = 1 to max_action_turn do
    for current_agent in agent_ids do
        env ← current_agent.perceive(env)
        (message, target_agent) ← current_agent.decide_action(env)
        dialogue_history ← EXECUTEACTION(current_agent, env, message, target_agent)
        env ← UpdateEnvironment(dialogue_history, env)
    end for
    discussion_summary ← SummarizeGroupMessages(env)
    env ← UpdateEnvironment(discussion_summary, env)
    is_complete ← TaskManager.CheckTaskCompletion(env)
    if is_complete then
        return env, ExtractTaskResult(env)
    end if
end for
return env, ExtractTaskResult(env)
```

to users; and History for storing records of conversations and insights gained during the interaction process.

D. Group Chat Orchestration

The group chat orchestration defines the complete lifecycle of collaborative problem-solving from initiation to conclusion. As shown in Algorithm 1, group chat is coordinated by Group Manager, following a preset maximum number of action turns.

Algorithm 1 demonstrates the core process of group chat. The system receives the maximum number of action turns, a list of participating agents, and the initial environment state as inputs. In each action turn, agents sequentially perform the complete process of perceiving the environment, making decisions, executing interactions, and updating the environment. At the end of each round, the system generates a dialogue summary and checks for task completion, returning results early if the task is completed. This structured process ensures orderly multi-agent collaboration and goal orientation, while providing a flexible task termination mechanism.

E. Agent Interaction

Agent interaction is the core element of group chat. Algorithm 2 describes the execution process of a single agent action, supporting both broadcast messages and directed communication modes.

Algorithm 2 implements flexible interaction modes between agents. This algorithm receives the initiating agent, current environment, message type, and target agent as inputs. The initiating agent first generates an initial message; if the recipient is all members, it directly returns the broadcast message record; otherwise, it enters a dual-agent alternating dialogue loop, which continues until the maximum number of turns is reached or the receiving party decides to terminate the dialogue. This mechanism allows agents to flexibly select interaction methods and objects based on task requirements, achieving multi-level collaboration from broad information sharing to in-depth professional discussion.

Algorithm 2 EXECUTEACTION{current_agent, env, message, target_agent}

```
Input: current_agent, env, message, target_agent, max_chat_turn
Output: dialogue_history
turn_count ← 1
dialogue_history ← [ ]
if target_agent is AllGroupMembers then
    message ← GenerateMessage(current_agent, env, message, target_agent)
    Append (current_agent, target_agent, message) to dialogue_history
    return dialogue_history
end if
sender ← current_agent
receiver ← target_agent
while turn_count < 2 · max_chat_turn do
    response ← GenerateResponse(receiver, dialogue_history, env)
    if not response then
        return dialogue_history
    end if
    Append (receiver, sender, response) to dialogue_history
    temp ← sender
    sender ← receiver
    receiver ← temp
    turn_count ← turn_count + 1
end while
return dialogue_history
```

F. Chat Results Processing

Chat results processing is a key link in ensuring effective transformation of multi-agent collaboration outcomes. Group Manager regularly summarizes dialogue content, extracting key information and phase conclusions, which are stored in agents' Memory to provide context support for subsequent interactions. The system simultaneously conducts quality assessment, examining whether the discussion has reached effective conclusions and whether it meets task requirements. Finally, the system performs format standardization processing on confirmed effective group chat results, forming structured task output to ensure compliance with Task Manager processing requirements.

V. EXPERIMENT SETUP

A. Task & Benchmark

1) Mathematical Reasoning: The experimental evaluation encompasses three established mathematical reasoning benchmarks: **GSM8K** comprises 1,300 test instances of grade school mathematics problems. **MATH** features 12,000 challenging problems spanning five difficulty levels, covering high school mathematics and competition-level mathematics. **AIME** collects American Invitational Mathematics Examination problems, featuring high-complexity mathematical problems from the prestigious competition, containing problems that assess advanced mathematical reasoning through exact match evaluation of numerical answers. All benchmarks employ symbolic equivalence verification for accuracy assessment.

2) Code Generation: The code generation capabilities are evaluated using two standardized benchmarks: **MBPP** (Mostly

Basic Python Problems) consists of 500 Python programming tasks accompanied by test cases, with performance assessed via unit-test pass rate. **HumanEval** provides 164 function-level programming challenges, evaluated through the pass@k metric:

$$\text{Pass@}k = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}} \tag{1}$$

where n denotes the total number of generated solutions and c represents correct solutions. We generate $n = 10$ candidate solutions per problem and compute the average pass rates at $k = 1/3/5$.

3) Domain-Specific Tasks: **FinQual** offers 1,000 multiple-choice and numerical calculation questions sampled from the CFA examinations and FinQA test sets, assessing financial reasoning and computational accuracy. **JEC-QA** contains 26,365 multiple-choice questions sourced from Chinese legal professional qualification exam preparation materials, evaluating legal concept understanding and scenario analysis. **MedmcQA** provides 194K multiple-choice questions from Indian medical entrance examinations, evaluating healthcare concepts and clinical reasoning.

4) Structural Text Understanding: **StrucText-Eval** assesses models' capabilities in understanding and processing structured textual data with varying complexity levels.

5) Commonsense Reasoning: **HellaSwag** offers 70,000 commonsense question-answering pairs formulated as contextual multiple-choice problems. **WinoGrande** provides 44,000 pronoun resolution challenges presented in a binary selection format, requiring disambiguation through contextual reasoning.

B. Baseline Methods

We evaluate five representative baseline methods:

Naive: Direct task forwarding to a single LLM without prompt engineering or problem decomposition. **Naive-CoT** Wei et al. (2022): Single-agent paradigm employing chain-of-thought prompting to elicit explicit reasoning traces. **ReAct** Yao et al. (2022): Structured single-agent framework decomposing problems through iterative Reasoning, Action, and Observation cycles. **AutoGen** Wu et al. (2023): Programmable multi-agent conversation framework with AssistantAgent and UserProxyAgent components. **Multi-Agent Debate** Liang et al. (2023): Collaborative paradigm where multiple agents analyze problems through predefined dialogue sequences until consensus.

C. Large Language Models

We employ two state-of-the-art LLMs: **Qwen2.5-72B-Instruct** and **Llama-3.1-70B-Instruct-Turbo**, selected for their superior performance in general-purpose task handling and computational efficiency.

VI. Experiment

A. Overall Performance

1) Mathematical Reasoning Analysis:

Finding 1

AgentGroupChat-V2 achieves superior mathematical reasoning performance through effective task decomposition and agent-level specialization, with the greatest advantages manifesting in high-complexity problems where structured collaborative analysis outperforms single-agent approaches.

Table I demonstrates AgentGroupChat-V2 's consistent superiority across mathematical reasoning benchmarks. On GSM8K, AgentGroupChat-V2 achieves 87.41% and 91.50% accuracy with Qwen2.5-72B and Llama-3.1-70B respectively. The performance gap becomes more pronounced on challenging tasks: on AIME(2024), AgentGroupChat-V2 reaches 30.4% accuracy with Qwen2.5-72B, nearly doubling other methods' performance. The substantial improvement from Naive (37.52%) to Naive-CoT (75.13%) on GSM8K highlights the importance of explicit reasoning, while ReAct's underperformance suggests its structured reasoning-action cycles may not align with pure mathematical reasoning demands.

2) Code Generation Analysis:

Finding 2

AgentGroupChat-V2 excels at generating high-quality initial solutions through collaborative analysis, achieving superior pass@1 performance, though structured frameworks like ReAct demonstrate greater effectiveness in diversified sampling scenarios, indicating task-specific architectural requirements.

Code generation results reveal AgentGroupChat-V2 's distinct performance characteristics. AgentGroupChat-V2 achieves optimal pass@1 scores: 79.20% on HumanEval with Llama-3.1-70B and 76.46% with Qwen2.5-72B. However, performance gains diminish at higher sampling rates (pass@5: 80.91% and 84.15%), contrasting with ReAct's improvement from 78.53% to 87.66%. This pattern suggests AgentGroupChat-V2 's collaborative mechanism rapidly produces high-quality solutions through multi-perspective analysis, but may limit exploration space in iterative optimization scenarios where diverse solution generation is crucial.

3) Commonsense Reasoning Analysis:

Finding 3

AgentGroupChat-V2 's task decomposition and multi-agent coordination create unnecessary overhead for commonsense reasoning, where direct model responses and simple chain-of-thought prove more effective than collaborative analysis.

TABLE I
EXPERIMENT RESULT ON MATH PROBLEM.

Baseline	LLM	GSM8K	MATH					AIME
			Level1	Level2	Level3	Level4	Level5	(2024)
Naive	Qwen2.5-72B	37.52	60.17	49.41	37.16	26.49	21.78	0.0
	Llama-3.1-70B	35.70	50.44	44.11	30.97	28.20	19.84	10.0
Naive-CoT	Qwen2.5-72B	75.13	85.84	77.65	75.22	63.68	48.25	10.0
	Llama-3.1-70B	87.33	91.15	88.82	87.17	80.77	67.32	16.7
ReAct	Qwen2.5-72B	52.76	53.10	35.88	36.73	28.21	19.46	3.3
	Llama-3.1-70B	20.09	28.32	11.76	7.52	4.27	2.33	0.0
AutoGen	Qwen2.5-72B	81.80	89.38	85.29	**88.05**	**71.79**	54.86	16.7
	Llama-3.1-70B	85.21	96.46	92.94	91.15	82.48	66.93	20.0
Multi-Agent Debate	Qwen2.5-72B	75.81	85.84	79.41	75.66	66.24	52.14	16.7
	Llama-3.1-70B	90.82	97.35	92.94	**92.48**	**85.90**	71.98	20.0
AGENTGROUPCHAT-V2	Qwen2.5-72B	**87.41**	**92.92**	**90.00**	84.07	**71.79**	**59.10**	**21.4**
	Llama-3.1-70B	**91.50**	**98.23**	**94.12**	88.94	81.20	**83.54**	**30.4**

TABLE II
EXPERIMENT RESULT ON CODE GENERATION.

Baseline	LLM	MBPP			HumanEval		
		pass@1	pass@3	pass@5	pass@1	pass@3	pass@5
Naive	Qwen2.5-72B	54.46	60.09	62.02	67.07	78.34	81.60
	Llama-3.1-70B	51.80	60.82	63.65	68.90	80.92	83.52
Naive-CoT	Qwen2.5-72B	51.30	61.52	65.08	62.86	78.60	82.43
	Llama-3.1-70B	53.04	60.51	62.75	71.58	81.23	83.56
ReAct	Qwen2.5-72B	57.02	**65.34**	**68.50**	75.54	**84.39**	**86.84**
	Llama-3.1-70B	55.88	62.39	64.84	78.53	**85.71**	**87.66**
AutoGen	Qwen2.5-72B	40.60	54.33	59.27	66.46	80.83	84.22
	Llama-3.1-70B	40.26	55.05	60.07	57.50	75.20	80.26
Multi-Agent Debate	Qwen2.5-72B	55.28	63.76	66.67	75.54	83.19	85.41
	Llama-3.1-70B	54.68	**63.02**	**65.75**	75.67	84.81	87.65
AGENTGROUPCHAT-V2	Qwen2.5-72B	**60.34**	63.76	64.45	**76.46**	82.31	84.15
	Llama-3.1-70B	**58.84**	60.35	60.84	**79.20**	80.38	80.91

Commonsense reasoning exposes AGENTGROUPCHAT-V2 's limitations when collaborative complexity exceeds task requirements. On HellaSwag, Naive methods achieve superior performance (73.7% vs AGENTGROUPCHAT-V2 's 70.3% on Qwen2.5-72B) because commonsense questions often have obvious correct answers that models can identify directly. AGENTGROUPCHAT-V2 's divide-and-conquer approach fragments these straightforward questions into unnecessary subtasks, while Naive-CoT's success (85.5% on WinoGrande with Qwen2.5-72B) demonstrates that single-model reasoning with explicit thought processes suffices.

Multiple agents discussing obvious answers can introduce doubt and second-guessing, leading to incorrect consensus where initial correct interpretations may be overturned through excessive deliberation. AutoGen's consistent underperformance (64.4% and 78.4% on Qwen2.5-72B) further confirms that structured multi-agent workflows become counterproductive when applied to tasks where immediate model knowledge access is most effective.

4) Structural Text Understanding Analysis:

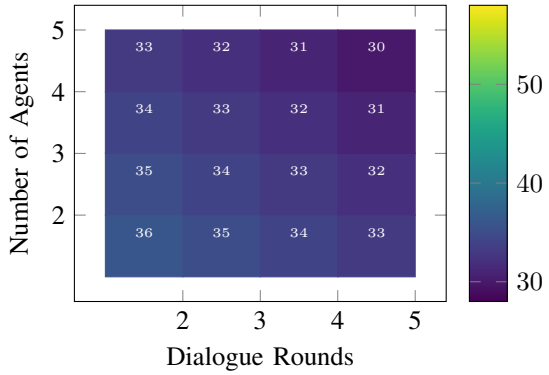

Fig. 4. AGENTGROUPCHAT-V2 w/ General Role performance on MATH-100

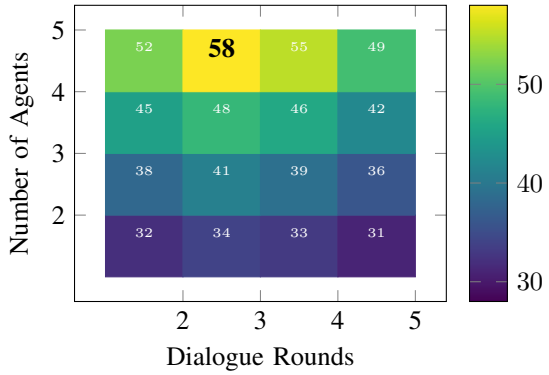

Fig. 5. AGENTGROUPCHAT-V2 w/ Specified Role performance on MATH-100

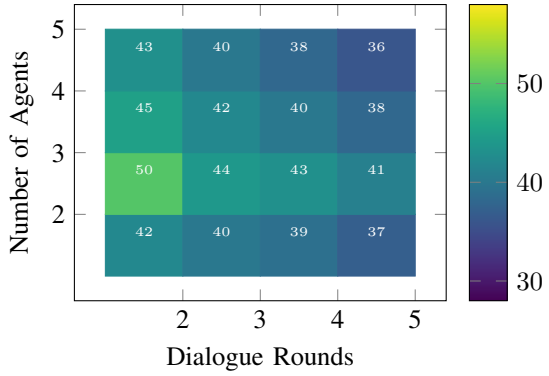

Fig. 6. AutoGen performance on MATH-100

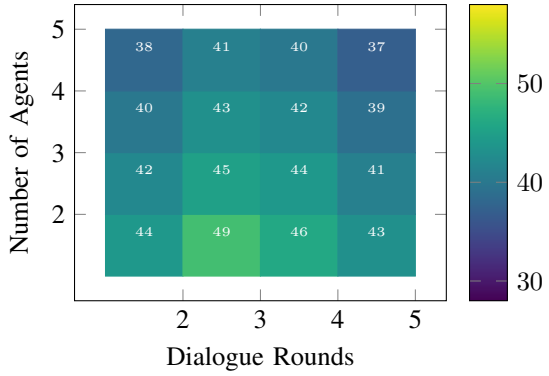

Fig. 7. Multi-Agent Debate performance on MATH-100

Finding 4

AGENTGROUPCHAT-V2 demonstrates robustness in handling complex structured information through effective task decomposition, maintaining performance stability as context complexity increases, while reasoning-intensive methods exhibit significant degradation under structured data challenges.

AGENTGROUPCHAT-V2 achieves optimal performance on high-complexity configurations, reaching 52.1% accuracy at Width=3, Depth=3, significantly outperforming other methods. As complexity increases, reasoning-intensive approaches show marked degradation: Multi-Agent Debate drops from 83.3% to 40.3%, and Naive-CoT from 86.7% to 42.3%. The direct Naive method maintains relative stability (45.0% on difficult configurations), while ReAct exhibits extreme performance collapse (below 1.2%). These results indicate that AGENTGROUPCHAT-V2 's divide-and-conquer strategy effectively manages structured data's context explosion challenges.

5) Domain-Specific Knowledge Analysis:

Finding 5

AGENTGROUPCHAT-V2 demonstrates domain-adaptive effectiveness, excelling in tasks requiring multi-perspective analysis (financial reasoning) while maintaining competitive performance across diverse professional domains, though complex interaction frameworks may introduce unnecessary overhead in knowledge-retrieval-intensive tasks.

Domain-specific results reveal AGENTGROUPCHAT-V2 's balanced performance across professional areas. In finance (FinQual), collaborative approaches achieve superior results with Multi-Agent Debate reaching 80.20%. Medical domain (MedmcQA) shows strong model dependency, with Llama-3.1-70B achieving 90.20% via Multi-Agent Debate. Legal reasoning (JEC-QA) challenges all methods, with maximum accuracy of 42.56%. AGENTGROUPCHAT-V2 maintains consistent performance across domains, while ReAct consistently underperforms, suggesting domain-specific architectural requirements for optimal effectiveness.

B. Ablation Study

To thoroughly understand the impact of different components in multi-agent systems on mathematical reasoning performance, we conducted comprehensive ablation experiments on the

TABLE III
EXPERIMENT RESULT ON COMMONSENSE REASONING.

Baselines	LLM	HellaSwag	WinoGrande
Naive	Qwen2.5-72B	**73.7**	80.3
	Llama-3.1-70B	**70.1**	82.4
Naive-CoT	Qwen2.5-72B	72.3	**85.5**
	Llama-3.1-70B	67.7	84.3
ReAct	Qwen2.5-72B	71.6	82.4
	Llama-3.1-70B	67.7	82.6
AutoGen	Qwen2.5-72B	64.4	78.4
	Llama-3.1-70B	61.1	73.1
Multi-Agent Debate	Qwen2.5-72B	72.5	85.0
	Llama-3.1-70B	68.9	85.1
AGENTGROUPCHAT-V2	Qwen2.5-72B	70.3	82.7
	Llama-3.1-70B	66.0	**85.6**

MATH-100 dataset. MATH-100 consists of 100 problems randomly sampled from Level 5 difficulty of the MATH dataset, specifically designed for parameter sensitivity analysis.

The experiments systematically evaluate the effects of two key dimensions—number of agents (2-5) and dialogue rounds (2-5)—on the performance of four different multi-agent frameworks. Additionally, we compare two distinct role configuration strategies in AGENTGROUPCHAT-V2 : general role configuration and specialized role configuration. As shown in Table VI, the general role configuration assigns all agents the same mathematical expert identity, while the specialized role configuration implements fine-grained division of labor based on the cognitive process of mathematical problem solving. All experiments use Qwen2.5-72B as the underlying language model.

1) Overall Performance Comparison: Heat map analysis from Figure 4 to Figure 7 reveals significant performance differences among the four methods. AGENTGROUPCHAT-V2 w/ Specified Role demonstrates the greatest performance potential across all configurations, achieving a maximum accuracy of 58% (5 agents, 3 dialogue rounds), while AGENTGROUPCHAT-V2 w/ General Role reaches only 36% (2 agents, 2 dialogue rounds). AutoGen achieves its best performance of 50% (3 agents, 2 dialogue rounds), and Multi-Agent Debate reaches 49% (2 agents, 3 dialogue rounds).

Finding 7

Specialized role division and homogeneous agent configuration exhibit completely opposite performance trends when the number of agents increases: the former improves with more agents while the latter deteriorates.

AGENTGROUPCHAT-V2 w/ Specified Role shows a strong positive correlation: accuracy significantly improves from an average of 32.5% with 2 agents to 53.5% with 5 agents, representing a 64.6% improvement. Each additional specialized agent contributes approximately 7 percentage points of performance gain. This consistent growth pattern indicates that specialized division of labor can avoid information redundancy through collaboration among different roles.

Conversely, AGENTGROUPCHAT-V2 w/ General Role exhibits a continuous performance decline as the number of agents increases: from an average accuracy of 34.5% with 2 agents to 31.5% with 5 agents, a decrease of 8.7%. When all agents assume the same mathematical expert role, adding more agents generates similar viewpoints and redundant information, leading to ineffective discussions rather than productive collaboration.

Finding 8

AGENTGROUPCHAT-V2 can effectively coordinate large-scale agent teams for collaboration, while traditional multi-agent frameworks suffer performance degradation as agent numbers increase, failing to achieve effective large-scale cooperation.

Both AutoGen and Multi-Agent Debate show performance decline with increasing agent numbers. AutoGen reaches its best average performance of 44.5% with 3 agents, then drops to 39.25% with 5 agents. Multi-Agent Debate achieves its highest average performance of 45.5% with 2 agents, then continuously declines to 39% with 5 agents. This suggests that traditional multi-agent frameworks cannot effectively manage interactions among more agents, resulting in reduced collaboration efficiency as scale increases.

In contrast, AGENTGROUPCHAT-V2 w/ Specified Role not only performs best in large-scale configurations but also exhibits the greatest performance potential range: 87.1% difference between optimal and worst configurations (58% vs 31%). Other methods show relatively limited performance ranges: AutoGen at 38.9%, Multi-Agent Debate at 35.1%, and AGENTGROUPCHAT-V2 w/ General Role at only 28.6%. This demonstrates that AGENTGROUPCHAT-V2 achieves more effective large-scale agent collaboration through specialized role division.

TABLE IV
EM ACCURACY FOR STRUCTEXT-EVAL.

Method	Model	Width=1			Width=2			Width=3		
		Depth=1	Depth=2	Depth=3	Depth=1	Depth=2	Depth=3	Depth=1	Depth=2	Depth=3
Naive	Llama3.1-70B	78.5	77.2	72.8	69.3	63.5	54.2	66.8	53.3	42.7
	Qwen2.5-72B	81.9	80.7	77.9	72.5	68.0	55.4	70.3	56.8	45.0
Naive-CoT	Llama3.1-70B	**84.1**	**83.8**	76.2	**75.5**	**70.3**	53.7	**73.2**	57.1	43.5
	Qwen2.5-72B	**86.7**	**85.3**	**78.3**	**77.8**	**71.4**	54.8	**75.5**	55.2	42.3
ReAct	Llama3.1-70B	5.2	3.8	2.1	4.1	2.9	1.3	3.5	1.8	0.5
	Qwen2.5-72B	8.1	6.7	4.9	6.3	4.2	2.8	5.1	3.1	1.2
AutoGen	Llama3.1-70B	28.4	24.7	19.8	22.1	17.3	12.6	18.9	13.2	8.7
	Qwen2.5-72B	34.2	31.1	26.5	27.8	22.9	17.4	24.3	18.1	12.8
Multi-Agent Debate	Llama3.1-70B	83.3	82.2	74.8	74.1	68.9	52.2	72.4	54.7	40.3
	Qwen2.5-72B	82.1	81.5	75.2	76.3	67.6	53.8	71.2	55.4	41.7
AGENTGROUPCHAT-V2	Llama3.1-70B	81.7	80.8	**79.1**	73.6	69.2	**58.4**	72.8	**62.9**	**48.7**
	Qwen2.5-72B	83.5	82.9	77.3	76.7	70.8	**59.2**	73.6	**64.7**	**52.1**

TABLE V
EXPERIMENT RESULTS ON LAW, FINANCE, AND MEDICAL MULTIPLE-CHOICE QA TASKS.

Method	Model	JEC-QA (Law) EM Accuracy	FinQual (Finance) EM Accuracy	MedmcQA (Medical) EM Accuracy
Naive	Llama-3.1-70B	31.58	71.83	73.90
	Qwen2.5-72B	42.56	76.00	89.30
Naive-CoT	Llama-3.1-70B	32.93	73.60	89.97
	Qwen2.5-72B	38.80	79.20	76.40
ReAct	Llama-3.1-70B	22.71	43.50	83.10
	Qwen2.5-72B	29.92	67.10	70.70
AutoGen	Llama-3.1-70B	30.08	63.76	56.79
	Qwen2.5-72B	40.60	79.16	68.60
Multi-Agent Debate	Llama-3.1-70B	31.58	78.28	90.20
	Qwen2.5-72B	38.50	80.20	75.90
AGENTGROUPCHAT-V2	Llama-3.1-70B	30.62	77.09	81.82
	Qwen2.5-72B	41.20	77.11	79.00

Finding 9

Specialized role division requires moderate interaction depth to integrate diverse perspectives, while homogeneous agents cannot achieve performance improvements by increasing dialogue rounds.

AGENTGROUPCHAT-V2 w/ Specified Role exhibits a pattern of initial increase followed by decrease: across most agent configurations, moderate interaction depth achieves optimal performance. For example, with 5 agents, accuracy rises from 52% at 2 rounds to 58% at 3 rounds, then drops to 49% at 5 rounds. This indicates that different specialized roles require sufficient interaction to integrate their professional insights, but excessive interaction can lead to decision complexity.

Conversely, AGENTGROUPCHAT-V2 w/ General Role is relatively insensitive to dialogue round variations, with accuracy slightly declining from 34.5% at 2 rounds to 31.5% at 5 rounds, a change of approximately 8.7%. Since all agents share the same professional background, extended dialogue mainly produces repetitive information and cannot gain new cognitive perspectives or quality improvements through increased interaction depth.

AutoGen performs best at 2 dialogue rounds, while Multi-

TABLE VI
ROLE CONFIGURATION FOR MULTI-AGENT MATHEMATICAL PROBLEM-SOLVING COLLABORATION.

Role Type	Role Setting
General Role	Agent-001 is a math expert.
Specialized Role	Agent-001 is a error detection specialist focused on identifying calculation mistakes, logical inconsistencies, and reasoning flaws in mathematical solutions.
	Agent-002 is a logical reasoning specialist specialized in mathematical proof construction, step-by-step deduction, and logical argument validation.
	Agent-003 is a context comprehension specialist responsible for understanding problem statements, extracting key information, and summarizing lengthy mathematical contexts.
	Agent-004 is a computational specialist dedicated to performing accurate calculations, numerical analysis, and algebraic manipulations.
	Agent-005 is a solution verification specialist focused on checking final answers, validating solution paths, and ensuring mathematical correctness.

Agent Debate reaches optimum at 3 rounds, both showing performance decline with further round increases, indicating that different collaboration mechanisms have their respective optimal interaction depth configurations.

VII. CASE STUDY

TABLE VII
AGENT ASSIGNMENT FOR INTERACTIVE DATA VISUALIZATION TOOL DEVELOPMENT TASKS

Task Name	Participating Agents
Module Interface Design	Requirement Analyst Agent, Code Implementation Agent, Code Review Agent
File Parsing Implementation	Requirement Analyst Agent, Code Implementation Agent, Code Review Agent
Data Processing Implementation	Requirement Analyst Agent, Code Implementation Agent, Code Review Agent
Data Display Implementation	Requirement Analyst Agent, Code Implementation Agent, Code Review Agent
Interactive Function Implementation	Requirement Analyst Agent, Code Implementation Agent, Code Review Agent
Testing and Verification	Test Planning Agent, Test Execution Agent, Quality Assurance Agent

A. Task Tree

To demonstrate AgentGroupChat's divide-and-conquer strategy, we present one representative scenarios illustrating how complex problems can be decomposed into sub-tasks and solved through specialized agent collaboration.

As illustrated in Figure 8, the interactive data visualization tool development is decomposed into four primary phases: module interface design, data processing functionality, visualization functionality, and testing verification. Upon completion of the module interface design, data processing and visualization functions can be developed in parallel, followed by unified testing verification. Each specific task is executed by specialized agents as listed in Table VII.

TABLE VIII
AGENT ASSIGNMENT FOR BLOCKCHAIN TECHNOLOGY ANALYSIS ARTICLE WRITING TASKS

Task Name	Participating Agents
Technology Survey	Research Planning Agent, Research Execution Agent, Content Review Agent
Case Collection	Research Planning Agent, Research Execution Agent, Content Review Agent
Market Analysis	Research Planning Agent, Research Execution Agent, Content Review Agent
Writing Technology Analysis Chapters	Writing Planning Agent, Writing Execution Agent, Content Review Agent
Writing Market Analysis Chapters	Writing Planning Agent, Writing Execution Agent, Content Review Agent
Overall Optimization	Integration Planning Agent, Integration Execution Agent, Quality Assurance Agent

Figure 9 demonstrates the workflow for composing a blockchain technology analysis article, which begins with three parallel research tasks: technology survey, case collection, and market analysis. Subsequently, based on the research findings, technical analysis and market analysis chapters are written in parallel, followed by overall optimization. As shown in Table VIII, each phase is configured with corresponding specialized agents to ensure content quality and professionalism.

B. Group Chat

To provide a concrete demonstration of inter-agent collaboration patterns, we present a file parsing task as an exemplar, illustrating how three specialized agents progressively refine solutions through two rounds of dialogue.

Through these two rounds of authentic dialogue, the collaboration pattern among the three agents becomes evident:

TABLE IX
TWO-ROUND AGENT DIALOGUE FOR FILE PARSING TASK

<table>
<tr><th>Round</th><th>Agent</th><th>Dialogue Content</th></tr>
<tr><td rowspan="3">Round 1</td><td>Requirement Analyst Agent</td><td>Let me analyze this file parsing task... We need to implement a parsing function supporting CSV and Excel files. Core requirements include: 1) File format validation; 2) Automatic character encoding detection; 3) Data parsing with JSON output; 4) Large file handling capability... I suggest implementing the basic framework first.</td></tr>
<tr><td>Code Implementation Agent</td><td><pre>
def parse_file(file_path):
    try:
        if file_path.endswith('.csv'):
            df = pd.read_csv(file_path)
        elif file_path.endswith('.xlsx'):
            df = pd.read_excel(file_path)
        else:
            return {"error": "Unsupported format"}
        # ... data processing logic
        return df.to_dict('records')
    except Exception as e:
        return {"error": str(e)}
</pre></td></tr>
<tr><td>Code Review Agent</td><td>Reviewing the implementation... The basic functionality exists, but several issues need attention: 1) No file size limitation, potential memory overflow; 2) Encoding detection relies entirely on pandas defaults, may cause garbled text; 3) Exception handling too simplistic, poor user experience; 4) Missing data type inference capability... Requires substantial improvements.</td></tr>
<tr><td rowspan="3">Round 2</td><td>Requirement Analyst Agent</td><td>Based on the code review feedback... Adding specific requirements: 1) 100MB file size limit; 2) Support UTF-8, GBK, ASCII encoding auto-detection; 3) Categorized error handling with user-friendly messages; 4) Intelligent data type recognition for dates, numbers, and text; 5) Chunked processing mechanism for large files... This should provide comprehensive coverage.</td></tr>
<tr><td>Code Implementation Agent</td><td><pre>
import chardet
import os
from typing import Callable, Dict, Any

class FileParser:
    def __init__(self, chunk_size=10000):
        self.chunk_size = chunk_size

    def parse_file(self, file_path: str) -> Dict[str, Any]:
        # File size validation
        if os.path.getsize(file_path) > 100*1024*1024:
            return {"error": "File exceeds 100MB limit"}

        # Enhanced encoding detection
        with open(file_path, 'rb') as f:
            raw_data = f.read(50000)
            detected = chardet.detect(raw_data)
            encoding = detected['encoding'] if detected['confidence'] > 0.7 else 'utf-8'

        try:
            if file_path.endswith('.csv'):
                # ... chunked CSV processing
                return self._parse_csv_chunked(file_path, encoding)
            elif file_path.endswith(('.xlsx', '.xls')):
                # ... Excel processing with compatibility
                return self._parse_excel(file_path)
        except Exception as e:
            return {"error": f"Parse error: {str(e)}", "type": type(e).__name__}

    def _parse_csv_chunked(self, file_path, encoding):
        # ... streaming processing implementation
        chunks = []
        for chunk in pd.read_csv(file_path, encoding=encoding, chunksize=self.chunk_size):
            chunks.append(chunk)
        df = pd.concat(chunks, ignore_index=True)
        return {"data": df.to_dict('records'), "dtypes": df.dtypes.to_dict()}
</pre></td></tr>
<tr><td>Code Review Agent</td><td>Final review assessment...</td></tr>
</table>

the Requirement Analyst Agent progressively transforms abstract requirements into technical specifications, the Code Implementation Agent iteratively develops solutions based on feedback, and the Code Review Agent ensures code quality from multiple perspectives.

VIII. CONCLUSION

This paper presents AGENTGROUPCHAT-V2 , a general-purpose framework for LLM-based multi-agent systems that systematically addresses critical challenges in the field through innovative architectural design and collaboration mechanisms. Our key contributions include a divide-and-conquer parallel architecture with hierarchical task forest structures, an adaptive collaboration engine for dynamic agent configuration, and optimized agent organization strategies—collectively enabling efficient distributed processing and superior performance across domains. Experimental results validate these contributions, with AGENTGROUPCHAT-V2 achieving substantial improvements on mathematical reasoning (91.50% on GSM8K, 30.4% on AIME), code generation (79.20% pass@1 on HumanEval), and various reasoning tasks. Most notably, performance advantages consistently increase with task complexity, demonstrating that AGENTGROUPCHAT-V2 's approach effectively leverages collective agent intelligence to solve challenging problems that

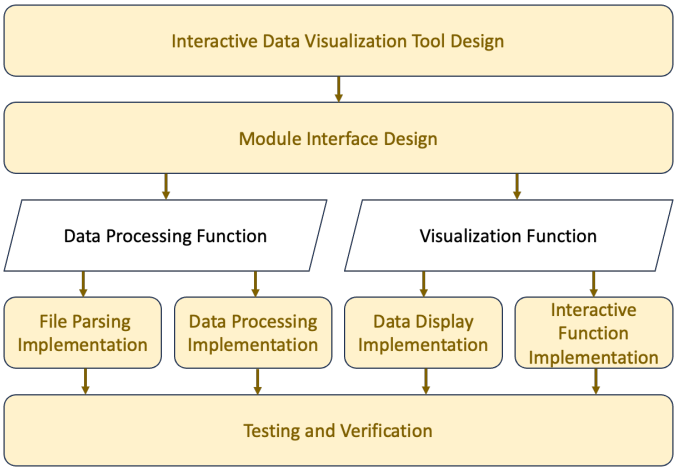

Fig. 8. A schematic illustration of task decomposition for the "Interactive Data Visualization Tool Development" task by AGENTGROUPCHAT-V2 . Rectangular boxes represent specific executable tasks, while parallelograms denote annotation labels for task groupings. Arrow connections indicate dependency relationships between tasks, and all tasks not directly connected can be executed in parallel after the completion of their prerequisite tasks.

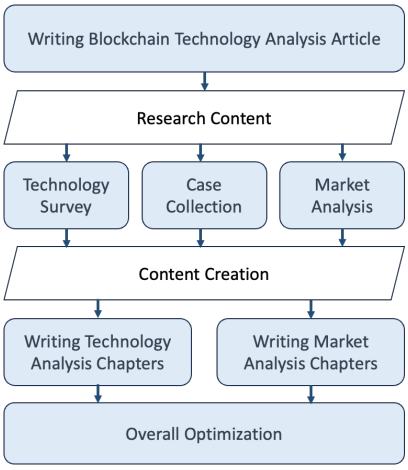

Fig. 9. Another illustration of task decomposition for "Writing Blockchain Techonology Analysis Article" task by AGENTGROUPCHAT-V2

exceed single-agent capabilities. By combining generalizability with computational efficiency and performance guarantees, AGENTGROUPCHAT-V2 establishes a new paradigm for practical, high-performance multi-agent systems capable of addressing increasingly complex real-world applications.

REFERENCES

Joseph Bates et al. The role of emotion in believable agents. *Communications of the ACM*, 37(7):122–125, 1994.

Mert Cemri, Melissa Z Pan, Shuyi Yang, Lakshya A Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, et al. Why do multi-agent llm systems fail? *arXiv preprint arXiv:2503.13657*, 2025.

Aili Chen, Xuyang Ge, Ziquan Fu, Yanghua Xiao, and Jiangjie Chen. Travelagent: An ai assistant for personalized travel planning, 2024. URL https://arxiv.org/abs/2409.08069.

Allan Dafoe, Yoram Bachrach, Gillian Hadfield, Eric Horvitz, Kate Larson, and Thore Graepel. Cooperative ai: machines must learn to find common ground, 2021.

Yilun Du, Shuang Li, Antonio Torralba, Joshua B Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate. *arXiv preprint arXiv:2305.14325*, 2023.

Jacques Ferber and Gerhard Weiss. *Multi-agent systems: an introduction to distributed artificial intelligence*, volume 1. Addison-wesley Reading, 1999.

Chen Gao, Xiaochong Lan, Zhihong Lu, Jinzhu Mao, Jinghua Piao, Huandong Wang, Depeng Jin, and Yong Li. S3: Social-network simulation system with large language model-empowered agents. *arXiv preprint arXiv:2307.14984*, 2023.

Chen Gao, Xiaochong Lan, Nian Li, Yuan Yuan, Jingtao Ding, Zhilun Zhou, Fengli Xu, and Yong Li. Large language models empowered agent-based modeling and simulation: A survey and perspectives. *Humanities and Social Sciences Communications*, 11(1):1–24, 2024.

Zhouhong Gu, Xiaoxuan Zhu, Haoran Guo, Lin Zhang, Yin Cai, Hao Shen, Jiangjie Chen, Zheyu Ye, Yifei Dai, Yan Gao, et al. Agentgroupchat: An interactive group chat simulacra for better eliciting emergent behavior. *arXiv preprint arXiv:2403.13433*, 2024.

Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Pei, Nitesh V Chawla, Olaf Wiest, and Xiangliang Zhang. Large language model based multi-agents: A survey of progress and challenges. *arXiv preprint arXiv:2402.01680*, 2024.

Xu Han, Zengqing Wu, and Chuan Xiao. "guinea pig trials" utilizing gpt: A novel smart agent-based modeling approach for studying firm competition and collusion. *arXiv preprint arXiv:2308.10974*, 2023.

Sirui Hong, Xiawu Zheng, Jonathan Chen, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, et al. Metagpt: Meta programming for multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352*, 2023.

Xu Huang, Weiwen Liu, Xiaolong Chen, Xingmei Wang, Hao Wang, Defu Lian, Yasheng Wang, Ruiming Tang, and Enhong Chen. Understanding the planning of llm agents: A survey, 2024. URL https://arxiv.org/abs/2402.02716.

Abhishek Kumar. Large language model based multi-agent system augmented complex event processing pipeline for internet of multimedia things. *arXiv preprint arXiv:2501.00906*, 2025.

Sanguk Lee, Tai-Quan Peng, Matthew H Goldberg, Seth A Rosenthal, John E Kotcher, Edward W Maibach, and Anthony Leiserowitz. Can large language models capture public opinion about global warming? an empirical assessment of algorithmic fidelity and bias. *arXiv preprint arXiv:2311.00217*, 2023.

Guohao Li, Hasan Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. Camel: Communicative agents for"

mind" exploration of large language model society. *Advances in Neural Information Processing Systems*, 36:51991–52008, 2023.

Tian Liang, Zhiwei He, Wenxiang Jiao, Xing Wang, Yan Wang, Rui Wang, Yujiu Yang, Shuming Shi, and Zhaopeng Tu. Encouraging divergent thinking in large language models through multi-agent debate. *arXiv preprint arXiv:2305.19118*, 2023.

Yaobo Liang, Chenfei Wu, Ting Song, Wenshan Wu, Yan Xia, Yu Liu, Yang Ou, Shuai Lu, Lei Ji, Shaoguang Mao, et al. Taskmatrix. ai: Completing tasks by connecting foundation models with millions of apis. *Intelligent Computing*, 3:0063, 2024.

Yuhan Liu, Xiuying Chen, Xiaoqing Zhang, Xing Gao, Ji Zhang, and Rui Yan. From skepticism to acceptance: Simulating the attitude dynamics toward fake news. *arXiv preprint arXiv:2403.09498*, 2024.

Junyu Luo et al. Large language model agent: A survey on methodology, applications and challenges. *arXiv preprint arXiv:2503.21460*, 2025.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35: 27730–27744, 2022.

Joon Sung Park, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*, pp. 1–22, 2023.

Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, et al. Chatdev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 15174–15186, 2024.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. Toolllm: Facilitating large language models to master 16000+ real-world apis, 2023. URL https://arxiv.org/abs/2307.16789.

Yoav Shoham and Kevin Leyton-Brown. *Multiagent systems: Algorithmic, game-theoretic, and logical foundations*. Cambridge University Press, 2008.

Jingyun Sun, Chengxiao Dai, Zhongze Luo, Yangbo Chang, and Yang Li. Lawluo: A chinese law firm co-run by llm agents. *arXiv preprint arXiv:2407.16252*, 2024.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.

Michael Wooldridge. *An introduction to multiagent systems*. John wiley & sons, 2009.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. Autogen: Enabling next-gen llm applications via multi-agent conversation framework. *arXiv preprint arXiv:2308.08155*, 2023.

Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, et al. The rise and potential of large language model based agents: A survey. *arXiv preprint arXiv:2309.07864*, 2023.

Chengxing Xie, Canyu Chen, Feiran Jia, Ziyu Ye, Kai Shu, Adel Bibi, Ziniu Hu, Philip Torr, Bernard Ghanem, and Guohao Li. Can large language model agents simulate human trust behaviors? *arXiv preprint arXiv:2402.04559*, 2024.

Kai Xiong, Xiao Ding, Yixin Cao, Ting Liu, and Bing Qin. Examining inter-consistency of large language models collaboration: An in-depth analysis via debate. *arXiv preprint arXiv:2305.11595*, 2023.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.

Jifan Yu, Zheyuan Zhang, Daniel Zhang-li, Shangqing Tu, Zhanxin Hao, Rui Miao Li, Haoxuan Li, Yuanchun Wang, Hanming Li, Linlu Gong, et al. From mooc to maic: Reshaping online teaching and learning through llm-driven agents. *arXiv preprint arXiv:2409.03512*, 2024.

Xinnong Zhang, Jiayu Lin, Libo Sun, Weihong Qi, Yihang Yang, Yue Chen, Hanjia Lyu, Xinyi Mou, Siming Chen, Jiebo Luo, et al. Electionsim: Massive population election simulation powered by large language model driven agents. *arXiv preprint arXiv:2410.20746*, 2024.

Yifan Zhang, Jingqin Wang, Jianye Yu, and Joey Tianyi Wen. Multi-agent reinforcement learning: A comprehensive survey. *arXiv preprint arXiv:2312.10256*, 2023.

Xuhui Zhou, Hao Zhu, Leena Mathur, Ruohong Zhang, Haofei Yu, Zhengyang Qi, Louis-Philippe Morency, Yonatan Bisk, Daniel Fried, Graham Neubig, et al. Sotopia: Interactive evaluation for social intelligence in language agents. *arXiv preprint arXiv:2310.11667*, 2023.

Yiyang Zhu. Multi-agent reinforcement learning in wireless distributed networks for 6g. *arXiv preprint arXiv:2502.05812*, 2025.